

环境配置文档

本环境配置文档用于配置一个容器来：

- 安装pgvector-baseline
- 安装我们的改良版pgvector
- 配置ANN-benchmark测试pgvector-baseline和我们的改进版本

以下的配置用于从头创建一个容器，你也可以联系 **刘宗熹** 2200013088@stu.pku.edu.cn 来获取我们配置完成的autodl容器。

在以下的操作中，我们假设你的工作目录是 {workspace}

安装pgvector-baseline

回到工作目录 `cd {workspace}`

运行 `git clone https://github.com/pgvector/pgvector.git pgvector-baseline`

安装我们的pgvector

运行 `https://github.com/mindtravel/pgvector.git`

切换到pgvector-gpu分支 `cd pgvector && git checkout pgvector-gpu`

配置ANN-benchmark测试各个版本的pgvector

环境配置

原版ann-benchmark是依靠在宿主机上为每个查询算法建立一个docker容器来测试的，但是目前我们的服务器本身就是一个docker容器。为了避免docker套docker的麻烦，且我们只需要测试pgvector_ivf，在测试不同版本的pgvector_ivf时，我们不再建立新容器，而是编译不同版本的vector.so扩展，并将其替换到postgresql中，然后运行ann-benchmark进行测试

具体配置步骤如下：

1. 拉取仓库

回到工作目录 `cd {workspace}`

运行 `git clone https://github.com/mindtravel/ann-benchmarks.git`

运行 `cd ann-benchmarks`

2. 创建并激活conda环境

运行 `conda create -n ann-benchmarks python==3.10.6`

运行 `conda activate ann-benchmarks`

3. 安装python依赖

运行 `pip install -r requirements.txt`

4. 安装各个ivf算法的依赖

原版pgvector：运行 `./scripts/env/pgvector_ivf.sh` 安装pgvector_ivf算法环境

我们的改进版本：运行 `./scripts/env/pgvector_ivf_gpu.sh` 安装pgvector_ivf_gpu算法环境

运行测试

测试脚本

测试脚本在目录 `scripts/tests` 中，各个脚本的内容如下：

- `compile.sh` :
 - `./scripts/tests/compile.sh baseline` 编译pgvector baseline
 - `./scripts/tests/compile.sh ours` 编译我们的改进版本
- `pgvector_origin.sh` : 对pgvector baseline进行单个测试（包含ivfflat多线程，ivfflat单线程，默认运行多线程版本）
- `pgvector_ours.sh` : 对我们的改进版本进行单个测试
- `test_baseline.sh` : 编译并测试pgvector baseline版本，自动下载三个数据集 **SIFT, DEEP, TEXT** 在所有数据集上运行所有测试并绘制图像到 `{workspace}/ann-benchmarks/results`
- `test_ours.sh` : 编译并测试我们的改进版本，自动下载三个数据集 **SIFT, DEEP, TEXT** 在所有数据集上运行所有测试并绘制图像到 `{workspace}/ann-benchmarks/results`

测试步骤

1. 回到工作目录 `cd {workspace}`，先修改 `./scripts/tests/compile.sh` 中 `vector.so` 的路径
2. 直接运行 `./scripts/tests/test_baseline.sh` 测试baseline版本
3. 直接运行 `./scripts/tests/test_ours.sh` 测试我们的版本
4. 运行完脚本后，命令行中会显示表格形式的测试结果，之后要查看这个表格结果，不需要重新运行 `test_all.sh`，只需要运行 `python plot.py --x-scale linear --y-scale log --batch`

改进版测试结果

1. SIFT-128 Euclidean

res	experience name	recall	qps	dpv
0	PGVector ivfflat Ours (lists=1000, probes=1)	0.377	7504.031	133.262
1	PGVector ivfflat Ours (lists=1000, probes=5)	0.755	12623.252	79.219
2	PGVector ivfflat Ours (lists=1000, probes=10)	0.879	14034.170	71.255
3	PGVector ivfflat Ours (lists=1000, probes=20)	0.955	12487.058	80.083
4	PGVector ivfflat Ours (lists=1000, probes=40)	0.988	10752.839	92.999

2. Deep-image-96 Angular

res	experience name	recall	qps	dpv
0	PGVector ivfflat Ours (lists=990, probes=1)	0.472	12658.668	78.997
1	PGVector ivfflat Ours (lists=990, probes=5)	0.830	19339.561	51.707
2	PGVector ivfflat Ours (lists=990, probes=10)	0.919	19893.973	50.266
3	PGVector ivfflat Ours (lists=990, probes=20)	0.968	17599.884	56.819
4	PGVector ivfflat Ours (lists=990, probes=40)	0.990	16001.227	62.495

3. Text-200 Angular

res	experience name	recall	qps	dpv
0	PGVector ivfflat Ours (lists=990, probes=1)	0.623	8367.508	119.510
1	PGVector ivfflat Ours (lists=990, probes=5)	0.918	14984.661	66.735
2	PGVector ivfflat Ours (lists=990, probes=10)	0.965	13018.782	76.812
3	PGVector ivfflat Ours (lists=990, probes=20)	0.987	13386.467	74.702
4	PGVector ivfflat Ours (lists=990, probes=40)	0.996	11615.774	86.090